

Telecom Complaint Priority Classification Pipeline

Urgency Engine x Severity Engine x Priority Decision Model

Technical Reference Document | Version 2.0 | August 2026

About This Document

This document describes the full end-to-end classification pipeline used by SignalCX to automatically assign a priority level (P1-P4) to every inbound telecom customer complaint. The system fuses two independent AI models -- an Urgency Engine and a Severity Engine -- using a weighted scoring formula to produce a single, actionable priority decision.

All weights, thresholds, formulas, and decision rules described here are extracted directly from the production source code (urgency_rules_v2.py, urgency_engine_v2.py, priority_model.py, priority_config.py).

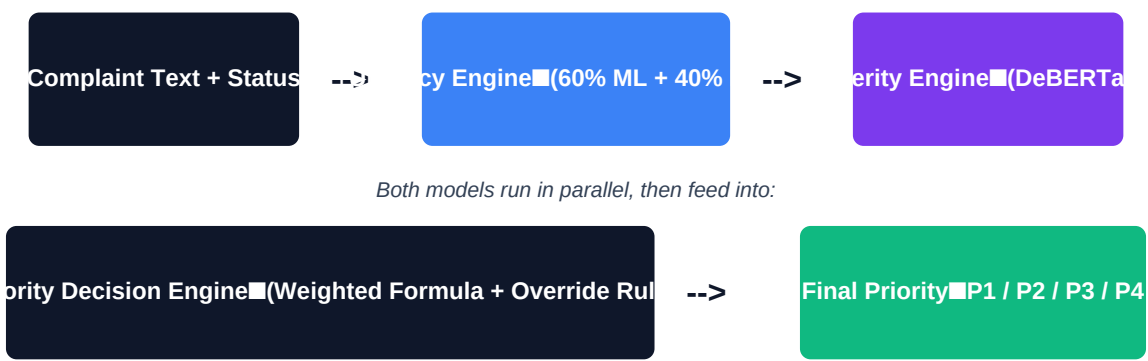
Priority Levels Produced by This Pipeline



Lower P-number = higher priority. P1 requires immediate escalation.

1. Pipeline Overview

When a customer complaint arrives, the system runs two independent classification models in parallel, then feeds their outputs into a Priority Decision Engine that produces the final P1-P4 label.



Key Outputs of the Pipeline

Output Field	Type	Description
urgency	Label	CRITICAL / HIGH / MEDIUM / NEUTRAL / LOW
urgency_score	Float [0, 1]	Normalised urgency score fed into priority formula
severity	Label	CRITICAL / HIGH / MEDIUM / LOW
severity_score	Float [0, 1]	Weighted probability score from DeBERTa-v2
priority	Label	Final decision: P1, P2, P3, or P4
priority_score	Float [0, 1]	Combined weighted score used for decision
priority_reason	String	Human-readable explanation of the decision

2. Urgency Engine (Hybrid: ML + Rule-Based)

The Urgency Engine calculates how time-sensitive and operationally urgent a complaint is. It combines a fine-tuned DistilBERT ML model with a deterministic rule engine to produce a hybrid urgency score in the range [-1.0, +1.0].

2.1 Rule-Based Engine (urgency_rules_v2.py)

The rule engine scans complaint text for 12 signal categories using regex pattern matching. Each detected signal contributes a fixed score. An Emergency fast-path bypasses all scoring.

Signal Category	Trigger Examples	Score Contribution
Emergency / Safety	911, fire, life-threatening, explosion, overheating, smoke	CRITICAL OVERRIDE -> rule_urgency = +1.0
Service Failure	no internet, network down, outage, cannot make calls, all drops	-0.50
Quality Problem	slow, poor network, throttled, speed is terrible, not as advertised	-0.20
Billing Issue	overcharged, double billed, incorrect charge, unauthorized charge, refund	-0.20
Serious Complaint	fraud, scam, deceptive, bait and switch, theft, unfair	+0.20
Duration	for 6 days, since last week, for months, for the past 6 weeks	+0.20
Repeated Support	contacted support 5 times, multiple calls, called customer service	+0.30
Unresolved / Persistent	still not fixed, nobody has fixed, ongoing, unresolved	+0.50 (if standalone)
Data Cap	data limit, 300GB cap, bandwidth limit, data allowance	+0.15
Account / Contract	cancellation, won't cancel, unauthorized account change	+0.10
Access Problem	cannot access my account, email access, blocking, cannot login	+0.10
Support Issue	technical support, customer service, tech support	+0.10
Active Case (status)	Status = Open or Pending	+0.02

Combination Bonuses (signal stacking):

Signal Combination	Additional Bonus
Service Failure + Duration	+0.10
Service Failure + Unresolved	+0.20
Duration + Unresolved	+0.05
Repeated Support + Unresolved	+0.05
Repeated Support + Duration	+0.05
Quality Problem + Duration	+0.10
Quality Problem + Repeated Support	+0.10

Data Cap + Billing	+0.10
Account Problem + Unresolved	+0.10
Serious Complaint + Unresolved	+0.10

Score is capped to [-1.0, +1.0] after all additions.

Rule Score to Label Mapping

Rule Score	Description
< -0.50	LOW -- Informational / no meaningful signal
-0.50 to +0.25	NEUTRAL -- Minor concern
> +0.25	HIGH -- Actionable urgency signal detected

2.2 ML Model -- DistilBERT (naman9705/signal-cx-urgency-distilbert)

A fine-tuned DistilBERT model classifies the complaint text into 4 urgency classes. The softmax probabilities are converted into a continuous score using fixed class weights.

Class Label	Class Weight	Interpretation
LOW (class 0)	-1.00	No urgency signal detected
MEDIUM (class 1)	-0.25	Minor concern, low priority
HIGH (class 2)	+0.50	Significant urgency
CRITICAL (class 3)	+1.00	Immediate action required

$$ml_urgency = (P_low \times -1.0) + (P_medium \times -0.25) + (P_high \times +0.50) + (P_critical \times +1.0)$$

ML Score to Final Urgency Label

ML Urgency Score	Final Urgency Label
< -0.50	LOW
-0.50 to +0.15	MEDIUM
+0.15 to +0.70	HIGH
> +0.70	CRITICAL
Emergency flag = True	CRITICAL (forced override)

2.3 Hybrid Fusion Formula

The final urgency score merges both sources. ML is weighted at 60% for semantic context; the rule engine at 40% for deterministic signal detection. Emergency flag forces override to 1.0.

Component	Weight	Rationale
-----------	--------	-----------

ML Model (DistilBERT)	60%	Understands semantic context and complaint tone
Rule Engine (regex signals)	40%	Explicit keyword detection, fast and interpretable

hybrid_urgency = (ml_urgency x 0.60) + (rule_urgency x 0.40)

If emergency_signal = True --> hybrid_urgency = 1.0 (forced override)

Score Normalisation (for Priority Formula Input)

Raw hybrid_urgency	Normalised urgency_score [0, 1]
-1.0	0.00 (minimum)
0.0	0.50 (neutral)
+1.0	1.00 (maximum)
Formula	normalised = (hybrid_urgency + 1.0) / 2.0

3. Severity Engine (DeBERTa-v2 Transformer)

The Severity Engine measures the impact severity of the complaint. It uses a fine-tuned DeBERTa-v2 transformer (703 MB local checkpoint) to produce a severity_score and label.

3.1 Model Details

Property	Value
Base Model	DeBERTa-v2 (Microsoft)
Task	Sequence Classification (4 classes)
Max Token Length	512 tokens
Device	CUDA if available, else CPU
Output	Softmax probabilities per class
Checkpoint Path	priority1/severity_outputs/severity_transformer/

3.2 Output Classes and Score Weights

The DeBERTa model outputs 4 classes. Probabilities are combined into severity_score:

Model Label	Canonical Label	Class Weight	Score Formula
Mild (id=0)	LOW	0.00	P_mild x 0.00
Moderate (id=1)	MEDIUM	0.33	P_moderate x 0.33
Severe (id=2)	HIGH	0.67	P_severe x 0.67
Critical (id=3)	CRITICAL	1.00	P_critical x 1.00

$$\text{severity_score} = (\text{P_mild} \times 0.00) + (\text{P_moderate} \times 0.33) + (\text{P_severe} \times 0.67) + (\text{P_critical} \times 1.00)$$

severity_score is in [0.0, 1.0] -- Higher = more severe impact

3.3 Severity Label and Score Mapping

Canonical Label	Fixed Score (label-based)	Mapped Priority
LOW (Mild)	0.00	P4
MEDIUM (Moderate)	0.50	P3
HIGH (Severe)	0.75	P2
CRITICAL (Critical)	1.00	P1

4. Priority Decision Engine

The Priority Decision Engine receives urgency and severity scores and labels, then applies a 7-step process to produce the final P1-P4 priority. A single CRITICAL signal always produces P1 regardless of numeric score.

4.1 Combined Score Formula and Weights

Component	Weight	Rationale
Severity Score	60% (0.60)	Impact on service quality dominates priority
Urgency Score	40% (0.40)	Time-sensitivity is a secondary factor

$$\text{combined_score} = (\text{severity_score} \times 0.60) + (\text{urgency_score} \times 0.40)$$

Both scores are normalised to [0.0, 1.0] before applying this formula.

4.2 Score-to-Priority Threshold Table

Combined Score	Score-Based Priority	Description
>= 0.85	P1 -- Critical	Immediate escalation. SLA breach imminent.
0.65 to 0.84	P2 -- High	Urgent. Resolve within hours.
0.40 to 0.64	P3 -- Medium	Significant. Resolve within 24 hours.
< 0.40	P4 -- Low	Informational. Resolve within standard SLA.

Source: priority_config.py -- THRESHOLD_P1=0.85, THRESHOLD_P2=0.65, THRESHOLD_P3=0.40

4.3 7-Step Decision Process

Step	Action	Detail
1	Run Urgency Engine	Call urgency_engine_v2.predict_urgency(complaint, status). Returns urgency_label and raw hybrid_urgency score in [-1, +1].
2	Run Severity Engine	Run DeBERTa-v2 model. Get severity_label (argmax) and severity_score (weighted probability sum).
3	Normalise Urgency Score	urgency_score = (hybrid_urgency + 1.0) / 2.0 -- maps [-1, +1] to [0, 1].
4	Compute Combined Score	combined_score = (severity_score x 0.60) + (urgency_score x 0.40)
5	*** CRITICAL Fast-Path Override ***	IF urgency_label = CRITICAL OR severity_label = CRITICAL --> final_priority = P1 immediately. Steps 6-7 are SKIPPED.

6	Score-Based Priority	Map combined_score to P1/P2/P3/P4 using the threshold table above.
7	Label-Based Escalation Guard	Convert each label to a priority. Take the HIGHEST priority (lowest P number) from: score_priority vs urgency_label_priority vs severity_label_priority. Ensures labels can only escalate, never downgrade.

Step 5 is the Critical Fast-Path. A single CRITICAL label forces P1 regardless of score.

5. Label-to-Priority Mapping Tables

Urgency Label	Label-based Priority
CRITICAL	P1 (Critical)
HIGH	P2 (High)
MEDIUM	P3 (Medium)
NEUTRAL	P4 (Low)
LOW	P4 (Low)

Severity Label	Score	Label-based Priority
CRITICAL	1.00	P1 (Critical)
HIGH	0.75	P2 (High)
MEDIUM	0.50	P3 (Medium)
LOW	0.00	P4 (Low)

6. Worked Examples

The following examples demonstrate how the pipeline classifies real complaints.

Example 1	P1 CRITICAL	"My router is on fire and I cannot call 911 - this is a life-threatening emergency!"
Signals Detected	Emergency keywords: 'fire', '911', 'life-threatening'	
Rule Urgency Score	+1.0 (Emergency fast-path)	
ML Urgency Score	~+1.0 (CRITICAL class)	
Hybrid Urgency (raw)	1.0 (Emergency flag forces +1.0)	
Urgency Label / Score	CRITICAL 1.00	
Severity Label / Score	CRITICAL 1.00	
Combined Score Calc	$(1.00 \times 0.60) + (1.00 \times 0.40) = 1.00$	
Final Decision	Step 5: CRITICAL urgency label --> P1 immediately (fast-path).	

Example 2	P1 CRITICAL	"Internet has been completely down for 10 days. I contacted support 5 times and nobody has fixed it."
Signals Detected	Service Failure (+0.50), Duration (+0.20), Repeated Support (+0.30), Unresolved (+0.50), Combo bonuses	
Rule Urgency Score	+1.0 (capped)	
ML Urgency Score	~+0.80 (CRITICAL class)	
Hybrid Urgency (raw)	~1.0	
Urgency Label / Score	CRITICAL 1.00	
Severity Label / Score	HIGH ~0.75	
Combined Score Calc	$(0.75 \times 0.60) + (1.00 \times 0.40) = 0.85$	
Final Decision	combined_score = 0.85 >= 0.85 threshold --> P1. Also CRITICAL urgency --> P1.	

Example 3	P2 HIGH	"I am being double-billed and Comcast refuses to process my refund. This is fraudulent."
Signals Detected	Billing (+0.20), Serious Complaint 'fraudulent' (+0.20), account+serious combo bonuses	
Rule Urgency Score	~+0.55	
ML Urgency Score	~+0.50 (HIGH class)	
Hybrid Urgency (raw)	~+0.53	
Urgency Label / Score	HIGH ~0.77	
Severity Label / Score	HIGH ~0.70	
Combined Score Calc	$(0.70 \times 0.60) + (0.77 \times 0.40) = 0.73$	
Final Decision	0.65 <= 0.73 < 0.85 --> P2 from score. HIGH urgency label --> P2. Final = P2.	

Example 4	P3 MEDIUM	"My internet speed has been slow since last week. It is not as advertised."
Signals Detected	Quality Problem (+0.30), Duration (+0.20), Combo quality+duration (+0.10)	
Rule Urgency Score	~+0.60	
ML Urgency Score	~+0.20 (MEDIUM-HIGH)	
Hybrid Urgency (raw)	~+0.44	
Urgency Label / Score	HIGH ~-0.72	
Severity Label / Score	MODERATE ~-0.40	
Combined Score Calc	$(0.40 \times 0.60) + (0.72 \times 0.40) = 0.53$	
Final Decision	0.40 <= 0.53 < 0.65 --> P3 from score. HIGH urgency label --> P2 override. Final = P2 (escalation gua	

Example 5	P4 LOW	"How do I change my Wi-Fi password?"
Signals Detected	No meaningful signal detected	
Rule Urgency Score	-0.70 (no signal fixed return)	
ML Urgency Score	~-0.80 (LOW class)	
Hybrid Urgency (raw)	$(-0.80 \times 0.60) + (-0.70 \times 0.40) = -0.76$	
Urgency Label / Score	LOW ~-0.12	
Severity Label / Score	LOW ~-0.05	
Combined Score Calc	$(0.05 \times 0.60) + (0.12 \times 0.40) = 0.08$	
Final Decision	0.08 < 0.40 --> P4. Both labels LOW --> P4. Final = P4.	

7. Complete Weight and Threshold Reference

7.1 Top-Level Priority Formula Weights

$$\text{combined_score} = (\text{severity_score} \times 0.60) + (\text{urgency_score} \times 0.40)$$

Source: `priority_config.py` -- `SEVERITY_WEIGHT=0.60`, `URGENCY_WEIGHT=0.40`

7.2 Urgency Engine Internal Weights

Sub-component	Weight	Source File
ML Model (DistilBERT)	60%	<code>urgency_engine_v2.py</code> line 220
Rule Engine (regex signals)	40%	<code>urgency_engine_v2.py</code> line 222
Emergency flag	Override: forces hybrid=1.0	<code>urgency_engine_v2.py</code> line 212

ML Class Weights inside DistilBERT Score:

Class	Weight in ml_urgency formula
LOW	-1.00
MEDIUM	-0.25
HIGH	+0.50
CRITICAL	+1.00

7.3 Severity Engine Probability Weights

Severity Class	Probability Weight
Mild -> LOW	0.00 (ignored)
Moderate -> MEDIUM	0.33
Severe -> HIGH	0.67
Critical -> CRITICAL	1.00

Source: `priority_config.py` -- `SEVERITY_CLASS_WEIGHTS`

7.4 Priority Decision Thresholds

Threshold Constant	Value	Result
THRESHOLD_P1	0.85	P1 -- Critical
THRESHOLD_P2	0.65	P2 -- High
THRESHOLD_P3	0.40	P3 -- Medium
(below all)	< 0.40	P4 -- Low

7.5 Urgency Score Normalisation

$$\text{urgency_score } [0,1] = (\text{hybrid_urgency} + 1.0) / 2.0$$

Source: `priority_config.py` -- `URGENCY_RAW_MIN=-1.0`, `URGENCY_RAW_MAX=+1.0`

7.6 Rule Engine Signal Base Scores

Signal	Score Added
Emergency (any keyword)	Override to +1.0 (skips all scoring)
Service Failure	+0.50
Unresolved (standalone)	+0.50
Quality Problem	+0.30
Repeated Support	+0.30
Billing	+0.20
Serious Complaint (fraud, scam)	+0.20
Duration	+0.20
Data Cap	+0.15
Account Problem	+0.10
Access Problem	+0.10
Support Issue	+0.10
Active Case (Open/Pending)	+0.02
No meaningful signal	-0.70 (fixed low return)

Quick Reference: The Complete Priority Formula

1. $\text{urgency_score} = (\text{hybrid_urgency} + 1.0) / 2.0$
2. $\text{hybrid_urgency} = (\text{ml_urgency} \times 0.60) + (\text{rule_urgency} \times 0.40)$
3. $\text{severity_score} = \text{sum}(\text{P_class} \times \text{class_weight})$ for [Mild=0, Moderate=0.33, Severe=0.67, Critical=1.0]
4. $\text{combined_score} = (\text{severity_score} \times 0.60) + (\text{urgency_score} \times 0.40)$

Priority Decision:

- P1 if $\text{combined_score} \geq 0.85$ (or any CRITICAL label override)
- P2 if $\text{combined_score} \geq 0.65$
- P3 if $\text{combined_score} \geq 0.40$
- P4 if $\text{combined_score} < 0.40$

Escalation Guard: Label-based priority can only escalate, never downgrade the score-based priority.